PROJECT REPORT

Generative Ai Course

Byte-to-Note

MEMBERS

A.I. Researcher: Martin Ferdinand Brandel

Data Engineering: Matheus Akl Barbalho Falqueto

ML Engineer: Andreea-Ioana Florea

Integration: Matteo Satanassi

Scrum Master: Siara-Caterina Cimponerescu, Souhail Izrhour

ABSTRACT + INTRODUCTION

Before starting to work on the project the team had one question: can a Large Language Model be trained purely on a natural language and learn through that to generate coherent songs if that music is printed as text? This is how we discovered that the ABC notation in music does offer us a simpler way to test and answer our big question. Our team ByteToNote has decided on a project that would evolve around a pretrained Language Model that would have the ability to generate melodies in the genre of Irish traditional music.

Our project uses an AI model called Qwen 3.5-0.8B and it's trained on a collection of Irish tunes written in ABC notation. Our goal was to both teach the model to create new songs on its own, but also to let the user choose the specific details for the tunes, for example if they want the key in D or G major, the time signature to either be 4/4 or 6/8 and how long the tune should be. The main idea for this project was to see if we could make an AI model made for giving a natural language output to learn to also "speak" in music. We thought this would prove to be a useful tool for finding inspiration and practice with new materials, also we have thought that such a project would make school music classes more engaging for children as today everything is digitalized and it would give music a total new different approach. However, it can also be used if the user simply wishes to explore how Irish music generated with AI sounds like.

In order to do all of this, we followed a few main steps: firstly, we have gathered and prepared a dataset of Irish tunes in ABC notation, making sure that the format was consistent for training, secondly, we fine-tuned the model on a shorter and cleaner dataset so that it learns the style and structure of the music. Afterwards we have tested the model to see if it can generate the tunes from the respected parameters, we have given: the key, meter and length requested by the user and whether the tune still sounded musically coherent. Along the way, we encountered some issues when generating the output, one of them being the fact that the model would sometimes copy existing tunes instead of creating new ones. We have realized that this issue occurred because there was not enough data the model trained on and that the songs in the dataset were very similar to one another.

This report is documenting all the methodologies used, technical trade-off's that were forced by our resource constraints and a final assessment of the output we got and the model's abilities and its limitations.

PROBLEM, OBJECTIVE AND PROJECT DEMONSTARTION

I. THE PROBLEM

Traditional music composition requires a large amount of musical knowledge, creativity, and often significant time to compose a complete and coherent melody. Although generative AI has made significant progress in text generation, applying language models to music remains challenging because music has its own structures, rules, and patterns. In particular, generating Irish music requires the model to understand elements such as notes, rhythm, meter, key, and musical structure while maintaining the characteristics of the Irish musical style.

Another challenging encounter we have faced is giving the user control over the generated music. A system that simply generates random melodies may not be useful if the user needs a specific key, time signature, or tune length. Therefore, the problem we wanted to address was whether a language model could learn the structure of Irish music represented in ABC notation and generate new tunes while following specific musical constraints provided by the user.

II. THE OBJECTIVE

The main objective of this project was to adapt a language model, Qwen 3.5-0.8B, to generate Irish tunes using ABC notation. We wanted the model to learn the patterns and characteristics of Irish music from an existing dataset and use this knowledge to generate new musical sequences.

A second objective was to give the user control over the generation process. The user should be able to specify characteristics such as the key, for example D or G major, the time signature, such as 4/4 or 6/8, and the desired length of the tune. The generated result should then respect these parameters while remaining structurally valid and musically coherent.

III.WHAT IS THE PROJECT IS TRYING TO DEMONSTRATE

Through this project, we wanted to demonstrate that a language model originally designed to process and generate text can be adapted to work with another structured form of information: musical notation. Since ABC notation represents music as text, it provides a way to experiment with music generation using techniques commonly associated with natural language processing.

The project also aims to demonstrate the potential and limitations of fine-tuning a general-purpose language model for a specialized creative task. In particular, we wanted to evaluate whether the model could learn the patterns of Irish music, follow user-defined constraints, and generate tunes that are sufficiently coherent and original rather than simply reproducing examples from the training dataset.

❖ A.I. RESEARCHER

As part of choosing the right model for this project, we looked into the comparisons between Qwen3.5-0.8B and other options to see what it offered, and we found several reasons why we should use it in our case.

First, it is essential that the data and ML engineers can work with the model as effectively as possible. The base model must therefore be relatively easy to understand, so the team can follow how data is used and how to fine-tune it afterwards by adapting it to our specific use case. Furthermore, the use of standard formats and compatibility with common ML frameworks enhances this usability, since it allows the team to work with more diverse datasets without extra integration overhead.

Second, one must also consider the end user's perspective as priority for how this tool could ideally be used going forward. Their main priority lies in what we would give the machine to generate a song, meaning they could use simple header fields such as the key, meter and length prefered for the song.

Qwen3.5 provides a native context length of 262,144 tokens, demonstrating the model's capacity to process a huge amount of information within a single sequence. This capacity matters

directly for our use case: generating ABC notation requires the model to track an entire piece's structure. It's worth noting that Qwen3.5 is also a genuinely multimodal model, supporting image and video input in addition to text. However, this project uses it exclusively in its text-generation capacity, since ABC notation is represented entirely as text, so while multimodality is a real capability of the base model, it isn't a feature this project draws on directly. While maximizing the model's capacity for adaptation is important to this project, we also had to work within the hardware available to us. The model needed to run on a regular consumer PC without exceeding its SSD, RAM, and VRAM limits. This constraint directly shaped our approach to fine-tuning.

Full fine-tuning means updating every parameter in the model during training. For Qwen3.5-0.8B, that for our project is 0.8 billion parameters, each requiring its own gradient computation and its own optimizer state (momentum and variance terms) tracked in memory throughout training. Even for a relatively small model, this adds up to a memory and compute requirement that wasn't realistic given our hardware, we had no dedicated high-end GPUs or cloud compute budget, so any approach assuming that kind of infrastructure was a non-starter from the outset. This is precisely why our fine-tuning approach needed to avoid disruptive modification of the core model and instead train selectively, focusing only on the parameters most relevant to our task. LoRA offered exactly this: rather than updating the entire model, it freezes all of the original pretrained weights and trains only a small set of newly added matrices layered onto specific parts of the network.

Beyond making training feasible on the hardware used for this project, this selective approach also reduces the risk of catastrophic forgetting and hallucination: because the core model's original weights are never modified, it remains far less likely that the model loses structural patterns it had already learned during pretraining. Weh ave had some challenges with the model trying to understand exactly what we wanted from it, even if the prompting was very specific, weh ave had moments in which it was giving us a every different output than what we expected. For example, once we tried to generate a new song, instead of receiving it, weh ave got a chess game. For the team and I, this was a challenge that weh ad to face and managed to give us the final and good output.

❖ DATA ENGINEERING

We cleaned and organized the dataset to make it suitable for training, starting with extracting key information from the ABC notations, such as the structural content of each piece. Alongside this, we added metrics such as note count, bar count and music length, which gave us a better picture of each sample's complexity, and duration, and it helped us identify the ones that didn't meet our quality standards. This preparation step was an important part of the data engineering pipeline supporting the LoRA fine-tuning, since the quality of the training data directly affects how well the model learns to generate valid and musically coherent ABC notations.

The dataset used for this project was the Irish Massive ABC Notation dataset, also known as IrishMAN, available on Hugging Face [1]. Working with music represented as text required us to consider both the organization of the data and the musical information contained in each sample. Our objective was to prepare examples that could support the model in learning the relationship between ABC symbols and musical sequences.

Extracting information from the samples also made the dataset easier to understand and inspect. Instead of working only with complete blocks of ABC text, we could examine additional characteristics of each piece. The extracted metrics provided different perspectives on the samples: note count helped describe the amount of musical content, bar count provided information about its organization, and music length helped us compare entries of different sizes. Looking at these characteristics together gave us a more useful overview than relying on a single metric.

As part of this process, we made sure to remove samples that were either empty or too short, since they didn't have enough musical content and wouldn't really add any value during the training process. We also filtered out samples with structural errors, for example mismatched bars, malformed notations or invalid characters, as a way to avoid teaching the model incorrect or unusable ABC patterns. We then checked the dataset for any duplicate tunes to reduce unnecessary repetition in the training data. This helped prevent certain examples from being overrepresented and lowered the risk of the model memorizing specific compositions instead of learning more general musical patterns, which mattered even more given that we were fine-tuning with LoRA rather than updating the full model.

For the short-sample filter, we used a minimum threshold of 10 characters in the musical body. This was a practical decision within the project to exclude extremely limited sequences. By examining the musical body rather than only the complete entry, we focused on the part of the sample that actually contained the music. An entry could contain text or formatting information without providing a sufficiently developed musical sequence for our training objective.

The different filters addressed different types of problems. Removing empty entries dealt with the absence of musical content, while filtering very short samples addressed the lack of context. Structural checks focused on the consistency of the notation, and duplicate checks addressed repetition across examples. Using these checks together allowed us to approach dataset quality from several perspectives rather than assuming that a sample was suitable simply because it contained text.

We also checked for unexpected or unwanted control codes, since these could interfere with the structure of the ABC notation and introduce patterns that weren't actually part of the musical content itself. On top of that, we verified that there was no overlap between the training and validation sets, making sure the same or at least highly similar samples didn't appear in both sets. This step was important in order to avoid any kind of data leak, which could make the model seem like it was performing better during the validation process than it actually would on new music.

The separation between training and validation was an important consideration because the two sets had different roles in the project. The training samples were used to adapt the model, while the validation samples provided a separate basis for assessing its behaviour during development. Keeping this distinction meaningful was necessary for interpreting the evaluation results and understanding whether the model could apply what it had learned to examples outside its training material.

Another important aspect of the preparation was balancing filtering with the preservation of useful musical content. The purpose was not to keep only pieces with the same length or structure, but to reduce identifiable problems while retaining variation among the remaining samples. Differences between pieces could still provide useful learning material, so the cleaning process focused on suitability for the project rather than making every entry identical in form.

After this cleaning and validation process, our final dataset contained 214,053 training samples and 2,162 validation samples, giving us a more consistent and reliable dataset to use for fine-tuning and evaluating the Qwen 3.5 model. Y

Together, the final training and validation sets contained 216,215 samples. Preparing this material connected the original dataset to the subsequent model-training stage. The extraction of information made the samples easier to examine, while the filtering and validation steps helped establish which entries would be retained for the experiments.

Overall, the data engineering work provided a foundation for the rest of ByteToNote. It supported both the preparation of training examples and the interpretation of later evaluation results. Although dataset cleaning alone could not guarantee the quality of the generated music, it helped us address problems in the input data before moving on to fine-tuning and generation.

❖ ML ENGINEER

The role of the Machine Learning Engineer is to build, deploy, train, fine tune, and maintain ML models in production. What the ABC notation does it encode the entire tune as ASCII[1], having the header fields of *X: index, M: meter, L: note length, K: key.* The letters from A to G are symbols that say in what key that the song would be written.

Because the melody, rhythm, and structure are a linear sequence of characters and nothing more, it can be made and modeled with the architecture used for natural language, meaning that a casual next-token-prediction[2] Large Language Model can have the capability to generate music in ABC notation. There is no need for any music architecture because the model *Qwen3.5-0.8B*[3]

[1] ASCII - *What is ASCII and how does it work? | Adobe Acrobat*. (2025). Adobe.Com. https://www.adobe.com/acrobat/resources/document-files/what-is-ascii.html

[2] QuarkAndCode. (2026, May). *Next-Token Prediction Explained: How LLMs Generate Text*. Medium. https://medium.com/@QuarkAndCode/next-token-prediction-explained-how-llms-generate-text-2851c5f71575

[3] https://huggingface.co/Qwen/Qwen3.5-2B

learns the statistical structure of the dataset. Cleaning the dataset was essential for us as it helped with training the model and not having errors at the generation time.

For the tokenization part of the project, we have used *Qwen's* tokenizer byte-pair-encoding,(BPE) `"tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, trust_remote_code=True)";` What it does is that common patterns get merged into single tokens if they were found frequently in the data. In our case, rare ABC specific symbol combinations may be split into multiple sub-tokens because the tokenizer was not designed for music notation. Another essential part that we needed to implement was the `"trust_remote_code=True";` because the model *Qwen* ships custom tokenizer implementation code on the Hugging Face hub more than reply on the standard transformers' library classes.

There was another challenge we have faced, that being that Qwen's tokenizer has no padding token, that being a placeholder added to much shorter text to make all the inputs in a batch of the same length for GPU processing. Thankfully, this was resolved with `"tokenizer.pad_token = tokenizer.eos_token";` That reuses the end of sequence token as a padding token. It helps avoid errors when batches of unequal length sequences need to have the same common length.

For the Load Base Model + *LoRA* weight precision was essential as every neural network is at its base core its just a number that needs to be stored somewhere in the memory. But *Qwen* 3.5-0.8B ships its weights in 16-bit floating point format or sometimes 32-bit. For a model with 0.8 billion parameters like ours, 16-bit precision requires 1.6 billion bytes, 1.5-1.6 GB just to hold the weights in memory before taking into consideration other factors like the gradients, optimizer states or the sequence of data. This was a big constraint for us as it can start to be more challenging once the training-time memory is added on top. So, in this case, `"load_in_4bit=True";` addresses this issue by sorting each weight using only 4 bits instead of 16. Researching about this issue we have found out that bits-and-bytes implements a scheme for quantization called NormalFloat4[4]. Due to time constraints, we have decided that NF4 could be implemented for future improvement since it is used for better accuracy. However, the reason why we implemented the `"bnb_4bit_compute_dtype=torch.bfloat16";` which governs what will happen when

[4] https://www.emergentmind.com/topics/normalfloat-4-nf4

a 4-bit weight will be used in a computation. In this case the "bnb" or bits-and-bytes does reconstruct a bfloat16 precision approximation from the stored 4-bit codes and their block scale factors. This helped with repeating independently on each layer for the whole duration of the training which ensures us that the operation run and a numerical and trustful precision.

Now for the device placement we have implemented `"device_map="auto";` because it does delegate the decision oof which device part of the model is placed on to the accelerate library which is essential because we did not want to manually assign the CPU/GPU. For the *LoRA* adapter attachment, what `"get_peft_model";` What it does is it wraps the quantized model with *LoRA*[5] (Low-Rank Adaptation) rather than fine tuning it at its full weights. For each of the targeted weights. *LoRA* freezes the *matrix W* and learns only two matrices A and B, so that the effective weight during the forward pass becomes *W+*. Because *rank r* is small compared to W's original size the number of the new trainable parameters is a very small fraction of what full fine-tuning[6] requires.

The *"r=16"* is the adapters rank. The value is a middle one as it balances the ability to learn the structure of ABC notation against the training cost and overfitting risk given the project's reduced dataset as we are using only 2% of it. Next, what we have done is we have set `"lora_alpha=32";` which is used very frequently to scale the *LoRA* updates magnitude without needing to have a specific hyperparameters. The `"task_type="CASUAL_LM";` arranges the adapter for autoregressive next token prediction. *LoRA* is applied to every of the attention projection.

In the fine-tuning part what it changes is the model's weights as it is shifted towards training on what is most likely to come next in an ABC notation tune. `"tokenized = dataset.map(lambda x: tokenizer(x["text"], truncation=True,`

---

[5] Zilliz. (2024, November 15). *LoRA Explained: Low-Rank Adaptation for Fine-Tuning LLMs*. Medium. https://medium.com/@zilliz_learn/lora-explained-low-rank-adaptation-for-fine-tuning-llms-066c9bdd0b32

[6] Black, S. (2026, August 20). *How to Fine-Tune an LLM: An End-to-End Guide*. Towards Data Science. https://towardsdatascience.com/how-to-fine-tune-an-llm-an-end-to-end-guide/

`max_length=512, padding="max_length"), batched=True”;` converts every tune's raw ABC text into the numeric token sequences the model trains on. The *“max_length=512”;* captures each sequence at 512 tokens and the reason we chose this length is because any tune that tokenizes longer gets cut off and because it would not waste too much to compute.

“Trainer” is handing the fetched batches, optimizing updates internally so none of that machinery had to be written by hand. `“per_device_train_batch_size=2”;` is kept at that size due to the GPU memory was a limiting factor into the project as if we would have put more, we would have run out of memory. The `“num_train_epochs=1”;` is restricting training to s single full pass over the data in case overfitting happens because the musical structure of the dataset songs is quite similar in some aspects. `“learning_rate=2e-4”;` is quite higher than the standard fine-tuning rates because in out project only the small and freshly initialized Laura adaptor parameters are used. For `“logging_steps=50”;` controls how often loss is logged in, so for every 50 steps producing the data I need it for one lost curve. Then there are checkpoints that are saved and at the end of each of the single epoch.

The data collator is building the training labels directly and automatically and it is essential for building the final training batched and building the labels Qwen learns from. The task is treated like the next token prediction. That leads to saying that the model is trained on the idea of “guessing what is coming next at each step”.

The challenges that we have had with the project, and the part of the ML engineering was updating each weight, which was very hard on the GPU memory because there were 428 data sets used as training examples. The limitations that we have had with the project was that it was not trained enough due to lack of hardware, as only 2% of the data set was trained, and due to time constraints. It was very interesting to see the changes of what the model was outputting us as we trained the model, furthermore, making it increasingly accurate. Because Quinn is bass trained on natural language. We have had some challenges with the model was understanding what exactly is needed from it, however, creating and building limitations in the fine-tuning part has led to qualitative results.

❖ INTEGRATION

To ensure a reproducible and collaborative development workflow, we initially established the project's technical foundation by configuring a standardized Python virtual environment (`.venv`) and structuring the central GitHub repository. This included managing project dependencies, defining version control practices, and establishing "`.gitignore`" rules for generated binary assets, MIDI files, and audio artifacts to streamline collaboration across the team.

Throughout the project lifecycle, we assisted each other with integration and technical support during each core development phase. It became also necessary to provide support for platform-specific dependencies and environment conflicts across Windows and macOS, resolving binary path configurations, execution bugs, and data schema discrepancies between sub-modules.

In the data engineering phase, we contributed to filtering and standardizing the training corpus. Because symbolic music models are highly sensitive to corrupted syntax, we designed automated data validation routines. These routines filtered out non-musical artifact tokens (such as generative tags and anomalous text headers), verified key signatures and time signatures, and guarded against corrupted measure boundaries that could destabilize tokenizer representations and training convergence.

To bridge the gap between the model's textual output and a functional audio demo, we developed a downstream post-processing pipeline centered on the music21 library. This pipeline entailed robust parsing of the raw ABC notation generated by our fine-tuned model, augmented by automated sanitization routines to resolve structural anomalies such as unbalanced repeat markers (|:, :|) and irregular measure lengths, that would otherwise cause runtime stream expansion crashes. Following this validation step, the parsed musical score was serialized into an intermediate standard MIDI file and subsequently rendered into a playable audio format using dedicated SoundFont banks (.sf2).

Engineering this conversion pipeline involved resolving significant integration hurdles within the Python environment, requiring extensive debugging to stabilize score parsing and audio playback across different runtime setups. While integrating FluidSynth as an external synthesis engine was initially implemented and evaluated for audio rendering, we ultimately decided against

including it in the final release. Relying on an external C-based binary introduced platform-dependent setup friction, particularly across different operating systems. Once the core music21 workflow was successfully stabilized, adopting a fully integrated Python-centric pipeline allowed us to maintain the project lightweight, portable, and straightforward to deploy.

Through this work, we connected the upstream data pipeline with the downstream inference output, delivering an end-to-end architecture capable of taking prompt-generated notation and producing audible music.

❖ SCRUM MASTER

The Scrum Master's main role was to guide the team throughout the project and make sure that the Scrum process was followed correctly, rather than directly doing the technical work. Our responsibility was mainly to help the team stay organized, understand what they needed to accomplish, and make sure that every member had a clear idea of their responsibilities. We tried to create an environment where everyone could work independently while still knowing that they had support whenever they needed it.

One of the main things we did was make sure that every team member understood what kind of research they needed to carry out according to their specific role. Instead of giving them direct answers, we tried to point them towards the right direction and suggest what they could search for. For example, if someone was not sure about a particular concept or task, we would suggest some topics or questions that could help them start their research. If we had simply given them the answers every time they encountered a problem, they might have completed their tasks faster, but they would not necessarily have understood the reasoning behind it. Instead, we encouraged them to investigate, compare different possibilities and then come back with what they had found.

Another important part of our role was following the progress of the team. We regularly checked with the different members to see how their work was progressing and whether they were facing any difficulties. These check-ins allowed us to understand whether everyone was moving

forward according to the project plan. By communicating regularly with the team, we could notice these situations and try to help. When we identified that someone was stuck, we tried to understand what exactly was causing the problem instead of giving them a solution. We asked questions about what they had already tried, and where they thought the problem was coming from. This helped us determine whether they needed technical guidance, more information, clarification about the task, or simply a different way of approaching the problem.

Another important aspect of the Scrum Master role was the organization, we had to keep an overall view of the project rather than focusing only on one specific technical task. This meant paying attention to what had already been completed, what was currently being worked on, and what still needed to be done while we also had to make sure that the team remained focused on the main objectives instead of spending too much time on tasks that were not priorities. We also had to make sure that the tasks were distributed clearly between the members. This was particularly important because the different parts of a project are often connected. If one person did not complete their part on time, it could potentially affect another member's work.

We also learned that every member of the team had a different way of working. For example, some people were comfortable working independently and only needed occasional guidance, while others preferred to discuss their progress more regularly or needed additional explanations before starting a task. Because of this, we could not use the same approach with everyone, so we had to adapt our communication depending on the person and the situation. Learning how to adapt our approach to different people was one of the most useful experiences we gained from this role.

As Scrum Master, we looked after the team's atmosphere, making sure to encourage each member to speak up about the problems that they were facing and to feel free to ask questions whenever they needed. We kept the motivation high by following their progress, recognizing the work that was completed and making sure to treat difficulties as a normal part of development so that it won't be seen as an issue that we can't overcome. This role taught us that organization means understanding the way that tasks depend on each other, not just making a schedule. We also learned to listen to their needs and adapt when things don't go as planned, asking questions like "What have you already tried?" to help members find solutions themselves.

CONCLUSION AND CHALLENGES

This project was made in order to answer a genuinely open question, that being if a model trained purely on natural language can learn to generate something as structured and creatively nuances as a song, simply by treating music as text. He final answer is yes, but definitely with some limitations and a certainty that it won't be able to replace the real musical creativity. Qwen 3.5-0.8B, fine-tuned via LoRA on ABC notation tunes has learned to reproduce the basic syntax and lose structural conventions of Irish traditional music, but the process made clear that technical competence in reproducing a format is not the same as a musical understanding. No matter how much or how well the model was trained, it couldn't compose with intent, only imitate patterns it had seen. There was no straightforward was of getting the result we desired, we kept having the same constraints, like limited hardware and time, including the careful balance of using just enough data so that the model doesn't overfit when generalizing. Despite these issues, the project was genuinely enjoyable to build, and it could have real potential as an educational tool if developed further.

Selecting this exact model and committing to LoRA as the fine-tuning method was a decision made in direct response to these resource constraints rather than an idealized best case. A model that was easy to understand and work with for the data and ML engineers, compatible with standard ML tooling and also small enough to actually fine-tune on consumer hardware was more important than chasing marginal capability gains from a bigger model we couldn't train. LoRA made the difference between a trainable project and an unusable one, we avoided prohibitive memory cost of full fine-tuning by freezing the base model and training only small adapter matrices.

Working downstream from the model's output was its own distinct challenge, functioning as an intensive exercise in defensive system engineering and full-pipeline integration. It exposed a fundamental tension between probabilistic text generation and rigid symbolic that could crash standard music parsers or send them into infinite loops. Addressing this required building automated sanitization routines and early-rejection filters rather than assuming the model's output would realizably conform to a valid schema, evaluating and ultimately deprecating complex

external binaries in favor of an integrated Python workflow forced a constant balancing act between theoretical output quality and practical deployment friction.

The hardest part of the project and training the model was to discover the ideal amount of data to use from the dataset due to the hardware limitations we had, because even with 2% of the data used it took more than one hour for the model to train. The most obvious issue we have faced that made us have these challenges was the actual output which sometimes was not accurate or just played random notes. If we would have had the ability to work with more of the dataset and give it more time to train, the music it created would have definitely been much more accurate in structure. Looking at the bigger picture, with the time, hardware and training limitations we have had the model had amazing output and we are satisfied with how the project turned out. However, one point all of the team has made that no matter how amazing a machine can create music, it does not have the same feeling as when a human plays it, concluding us to the fact that such projects can be great for learning but not performing.

Bibliography

1. Packowski, S. (2024, December 14). *Natural language interfaces powered by large language models*. Medium. https://sarah-packowski.medium.com/natural-language-interfaces-powered-by-large-language-models-e2fe04cc541e
2. Benini, D. (2025). *ABC notation, the basics - Nota ABC*. Notabc.App. https://notabc.app/abc/basics/
3. Ferrer, J. (2024, February 14). *An Introductory Guide to Fine-Tuning LLMs*. Datacamp.Com; DataCamp. https://www.datacamp.com/tutorial/fine-tuning-large-language-models
4. Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. de L., Hendricks, L. A., Johannes, W., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Bogdan, D., Guy, A., Osindero, S., Simonyan, K., Elsen, E., … Laurent, S. (2022). *Training Compute-Optimal Large Language Models* (2203.15556v1). arXiv. https://arxiv.org/abs/2203.15556v1
5. *What is music21? — music21 Documentation*. (2026). Music21.Org. https://music21.org/music21docs/about/what.html
6. Mark Hennings. (2023, December 14). *LoRA & QLoRA Fine-tuning Explained In-Depth*. YouTube. https://www.youtube.com/watch?v=t1caDsMzWBk

7. GeeksforGeeks. (2024, August 2). *Generating Music Using ABC Notation*. GeeksforGeeks. https://www.geeksforgeeks.org/nlp/generating-music-using-abc-notation/
8. Ferrer, J. (2024, February 14). *An Introductory Guide to Fine-Tuning LLMs*. Datacamp.Com; DataCamp. https://www.datacamp.com/tutorial/fine-tuning-large-language-models
9. *FluidSynth | Software synthesizer based on the SoundFont 2 specifications*. (n.d.). Www.Fluidsynth.Org.
10. https://huggingface.co/docs/transformers/en/models
11. sander-wood. Irish Massive ABC Notation (IrishMAN). Hugging Face. https://huggingface.co/datasets/sander-wood/irishman
12. Official reference for ABC notation syntax and structure, supporting the discussion of musical information extraction and structural checks. https://abcnotation.com
13. Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., & Carlini, N. (2022). Deduplicating Training Data Makes Language Models Better. *Proceedings of the 60Th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. https://doi.org/10.18653/v1/2022.acl-long.577
14. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685 [Cs]*. https://arxiv.org/abs/2106.09685
15. Belcic, I., & Stryker, C. (2025, April). *Catastropic forgetting*. Ibm.Com. https://www.ibm.com/think/topics/catastrophic-forgetting
16. *What is Multimodal AI? | Stanford HAI*. (2025). Stanford.Edu. https://hai.stanford.edu/ai-definitions/what-is-multimodal-ai

ANNEX

```
tokenizer.save_pretrained(OUTPUT)
```

✓ [5] 31m 46s

```
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8]    function: '_dequantize_4bit_compute'
(/Users/andreeaioanaflorea/PycharmProjects/Project-ByteToNote/.venv/lib/python3.14/site-packages/bitsandbytes/backends/default/ops.py:262)
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8]    last reason: 1/7: shape[0] == 2048
                                  # n = prod(shape)  # bitsandbytes/backends/default/ops.py:271 in _dequantize_4bit_compute
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8] User stack trace:
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8]   File
"/Users/andreeaioanaflorea/PycharmProjects/Project-ByteToNote/.venv/lib/python3.14/site-packages/bitsandbytes/backends/default/ops.py", line 271, in
 _dequantize_4bit_compute
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8]     n = prod(shape)
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8] To log all recompilation reasons, use
TORCH_LOGS="recompiles".
W0918 08:08:00.822000 25211 .venv/lib/python3.14/site-packages/torch/_dynamo/convert_frame.py:2008] [1/8] To diagnose recompilation issues, see
https://docs.pytorch.org/docs/main/user_guide/torch_compiler/compile/programming_model.recompilation.html
```

[535/535 31:35, Epoch 1/1]

Step	Training Loss
50	2.020712
100	1.662280
150	1.629788
200	1.553571
250	1.533263
300	1.523082
350	1.526364
400	1.424946
450	1.448067
500	1.389205

```
('lora-out/tokenizer_config.json',
 'lora-out/chat_template.jinja',
 'lora-out/tokenizer.json')
```

```
        learning_rate=2e-4,
        logging_steps=50,
        save_strategy="epoch",
        bf16=True,
        report_to="none",
    ),
    train_dataset=tokenized,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False),
)
trainer.train()

model.save_pretrained("lora-out")
tokenizer.save_pretrained("lora-out")
```

✓ [27] 18m 20s

```
Map: 100%|██████████| 428/428 [00:00<00:00, 2631.68 examples/s]
```

[214/214 18:06, Epoch 1/1]

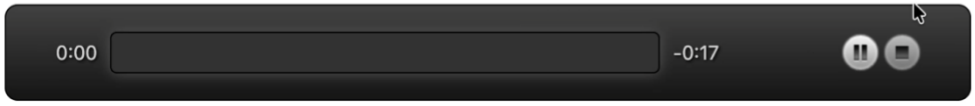
Step	Training Loss
50	2.097204
100	1.778041
150	1.549086
200	1.507751

```
('lora-out/tokenizer_config.json',
 'lora-out/chat_template.jinja',
 'lora-out/tokenizer.json')
```

```
generate("C", "1/...
```

✓ [7] 16s 946ms

```
X:1
L:1/8
M:1/4
K:C
(3(3C2C) | c2 C/B/A/G | F2 G/E/D/C | D2 A/c/d/e/f | g3 e | d2 f/g/a/f | e2 z B | c2 z c |
 B2 z B | c2 z c | B2 z B | c2 z B | c2 z c | B2 z B | c2 z B | c2 z B | c2 z B|]
```




```python
key = "D"     key
meter = "3/4"
note_length = "4/4"
bars = 16

prompt = f"X:1\nL:{note_length}\nM:{meter}\nK:{key}\n"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

out = model.generate(**inputs, max_new_tokens=300, do_sample=True, temperature=0.9, top_p=0.95, pad_token_id=tokenizer.eos_token_id)
text = tokenizer.decode(out[0], skip_special_tokens=True)

# Cut off at the Nth bar line so length roughly matches what was asked for.
# Small models can't reliably count bars themselves, so this is done by hand.
parts = text.split("|")
kept = parts[: bars + 1]  # +1 accounts for the header taking the first "cell"
print("|".join(kept))
```

✓ [32] 1m 52s

```
X:1
L:4/4
M:3/4
K:D
|: A3 D3 F3 | A2 B2 c2 d2 | A3 D3 F3 | G2 F2 E2 d2 | A3 D3 F3 | E2 F2 D2 c2 | d2 B2 A2 g2 |
 f2 g2 f2 e2 | d2 c2 B2 A2 | G2 D2 A2 G2 | A2 G2 G2 G2 | f2 e2 d2 B2 | f2 g2 f2 e2 |
 d2 c2 B2 A2 | G2 D2 A2 G2 | A2 F2 F2 F2
```